



SOFTWARE ENGINEERING

C03001

CHAPTER 7 — ARCHITECTURAL DESIGN

Anh Nguyen-Duc
Tho Quan-Thanh



Adapted from <https://iansommerville.com/software-engineering-book/slides/>

TOPICS COVERED

- ✓ Architectural design decisions
- ✓ Architectural views
- ✓ Architectural patterns
- ✓ Application architectures

SOFTWARE ARCHITECTURE

- ✓ Describes how the system is organized as a set of communicating components

Architecture?

"Architecture" can mean: (<http://en.wikipedia.org/wiki/Architecture>)

A general term to describe buildings and other physical structures.

The art and science of designing buildings and (some) non-building structures.

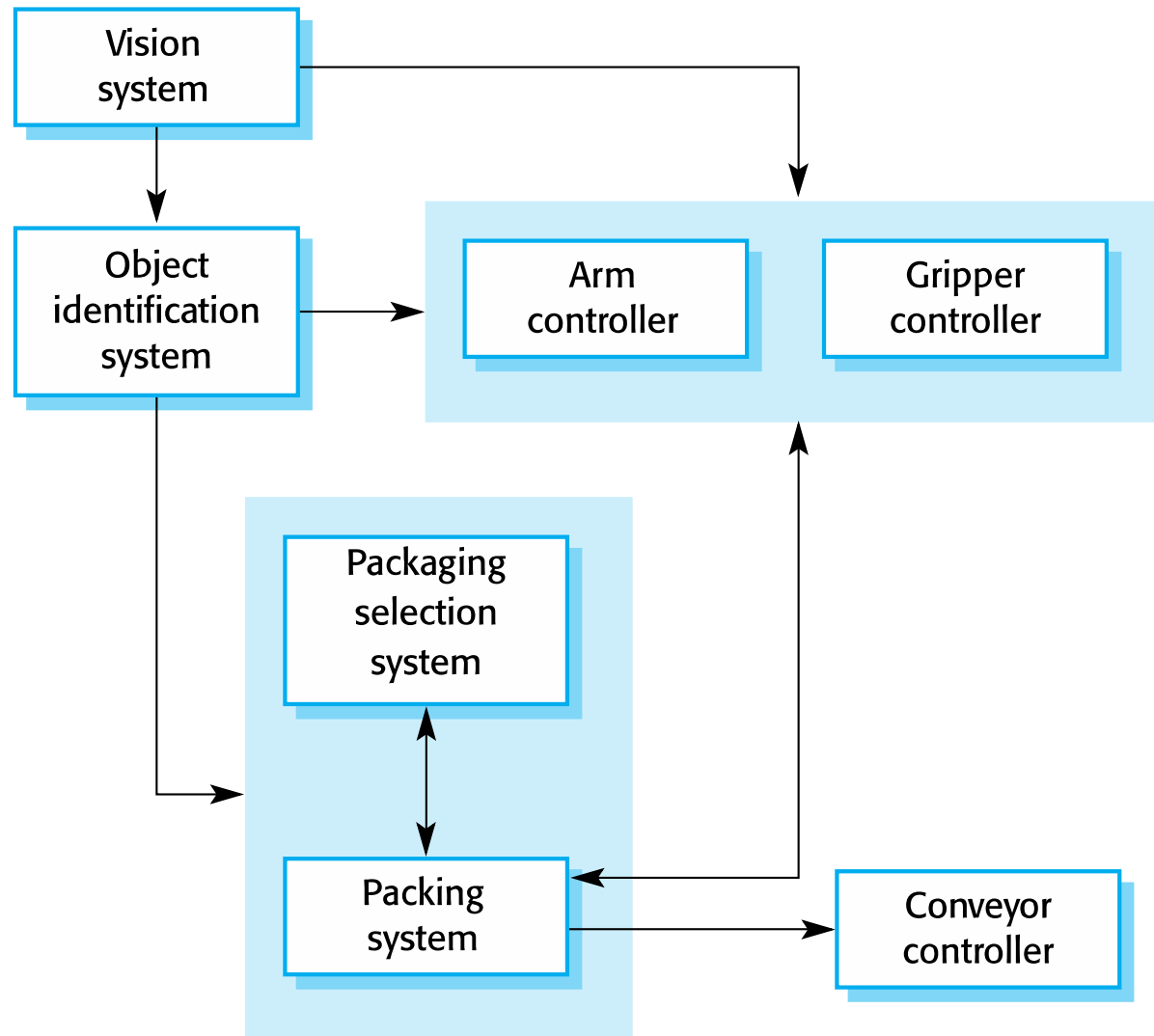
The style of design and method of construction of buildings and other physical structures.

...

ARCHITECTURAL DESIGN

- ✓ Concerned with:
 - how a software system should be organized and designing the overall structure of that system.
 - “build-a-house” metaphor: front end, back end, etc
 - “gardening” metaphor: refactoring, maintenance, etc
- ✓ The critical link between design and requirements engineering
 - as it identifies the main structural components in a system and the relationships between them.
- ✓ Agile?
 - An early stage: design an overall systems architecture.
 - Refactoring the system architecture is usually expensive

THE ARCHITECTURE OF A PACKING ROBOT CONTROL SYSTEM



ARCHITECTURAL ABSTRACTION

- ✓ Architecture in the small
 - is concerned with the architecture of individual programs.
- ✓ Architecture in the large
 - is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components.

Software architecture is important because it affects the performance, robustness, distributability, and maintainability of a system.

- Func. reqs.: individual components
- Non-func. reqs: depend on the system architecture

ADVANTAGES OF EXPLICIT ARCHITECTURE

- ✓ Stakeholder communication
 - Architecture may be used as a focus of discussion by system stakeholders.
- ✓ System analysis
 - Means that analysis of whether the system can meet its non-functional requirements is possible.
- ✓ Large-scale reuse
 - The architecture may be reusable across a range of systems
 - Product-line architectures may be developed.

ARCHITECTURAL REPRESENTATIONS

- ✓ Mostly use: Simple, informal block diagrams showing entities and relationships
 - Problem: lack semantics
- ✓ Box and line diagrams
 - Very abstract - do not show the nature of component relationships nor the externally visible properties of the sub-systems.
 - However, useful for communication with stakeholders and for project planning.

USE OF ARCHITECTURAL MODELS

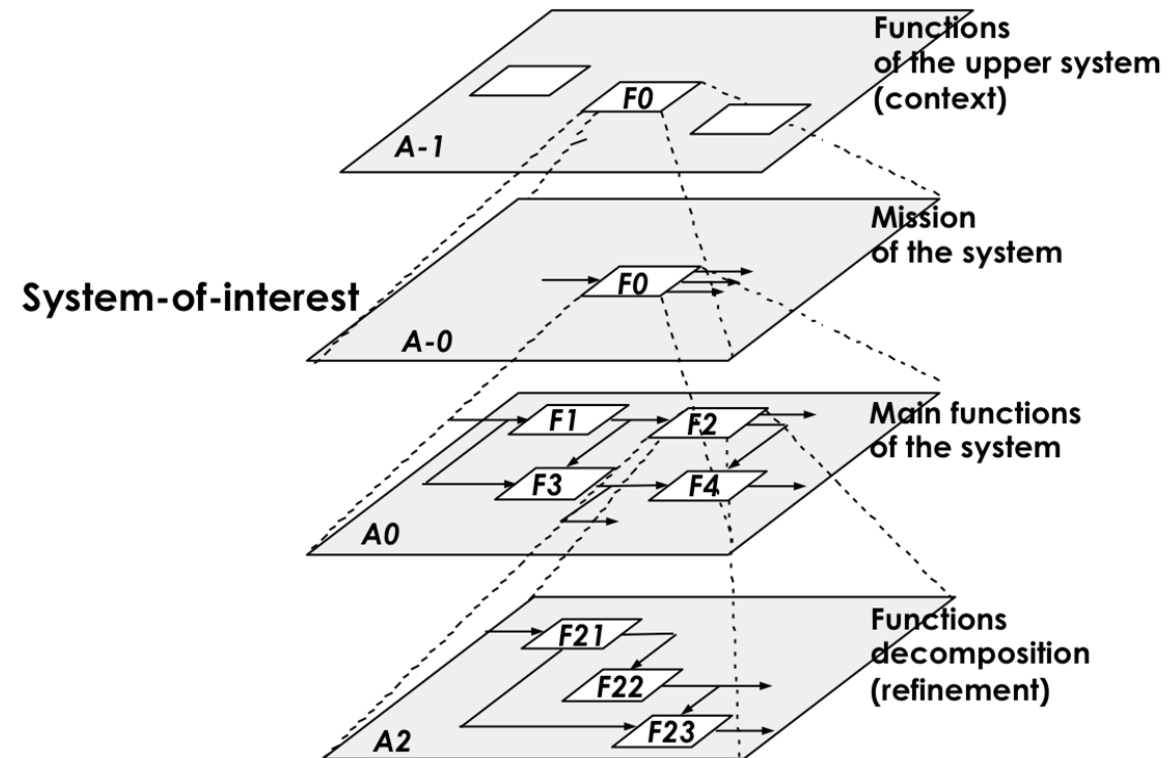
- ✓ As a way of facilitating discussion about the system design
 - A high-level architectural view is useful for communication with system stakeholders and project planning
- ✓ As a way of documenting an architecture that has been designed
 - to produce a complete system model that shows the different components in a system, their interfaces and their connections.

ARCHITECTURE DECOMPOSITION

- ✓ Software systems:
 - complexity problem $\leq$ inter-relationship
- ✓ Goals:
 - Maximizing cohesion
 - Minimizing coupling

Cohesion: degree of communication taken place among the module's elements

Coupling: degree of communication among modules

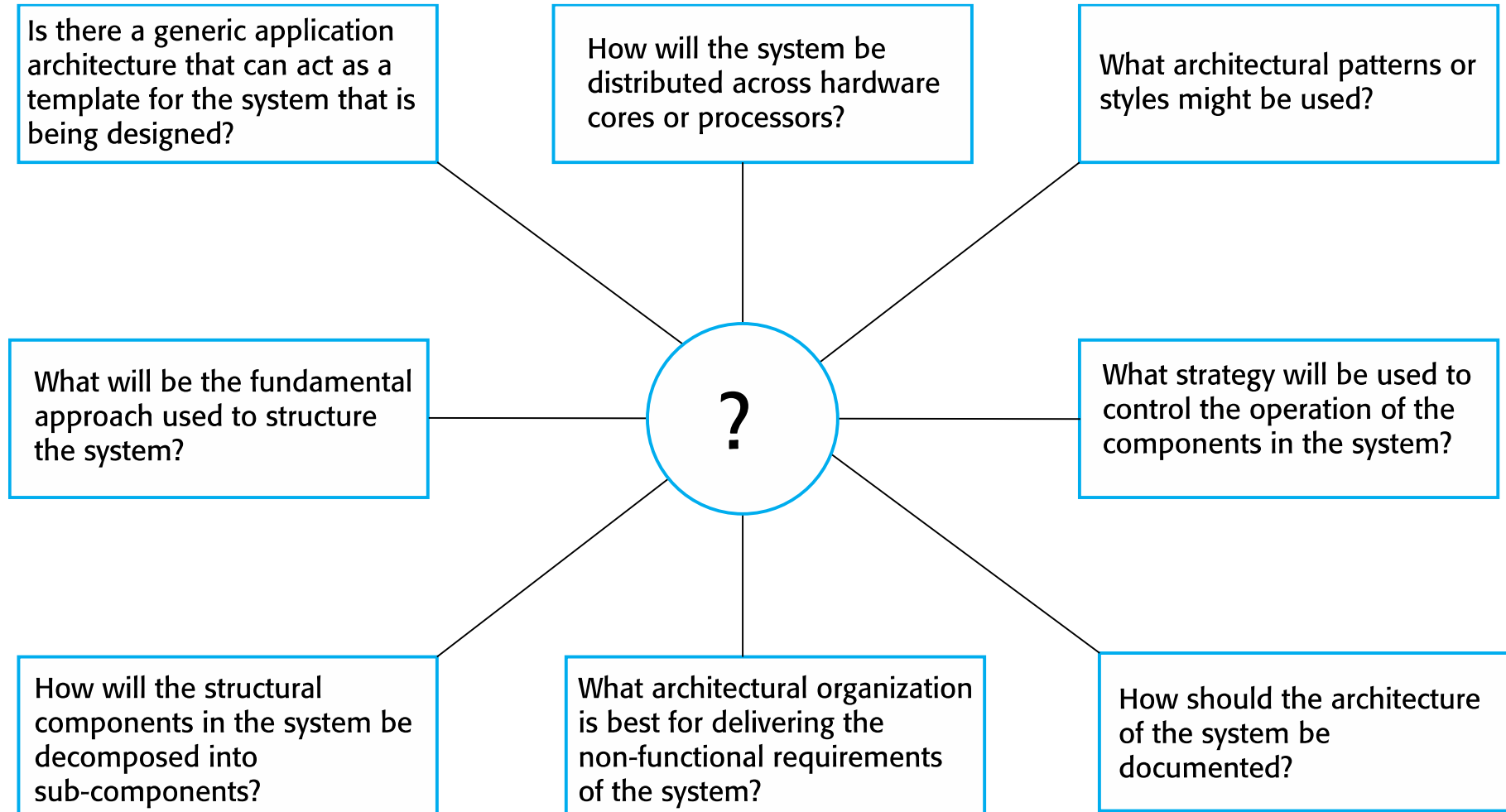




ARCHITECTURAL DESIGN DECISIONS



ARCHITECTURAL DESIGN DECISIONS



ARCHITECTURE AND SYSTEM CHARACTERISTICS

- ✓ **Performance**
 - Localize critical operations and minimize communications. Use large rather than fine-grain components.
- ✓ **Security**
 - Use a layered architecture with critical assets in the inner layers.
- ✓ **Safety**
 - Localize safety-critical features in a small number of sub-systems.
- ✓ **Availability**
 - Include redundant components and mechanisms for fault tolerance.
- ✓ **Maintainability**
 - Use fine-grain, replaceable components.



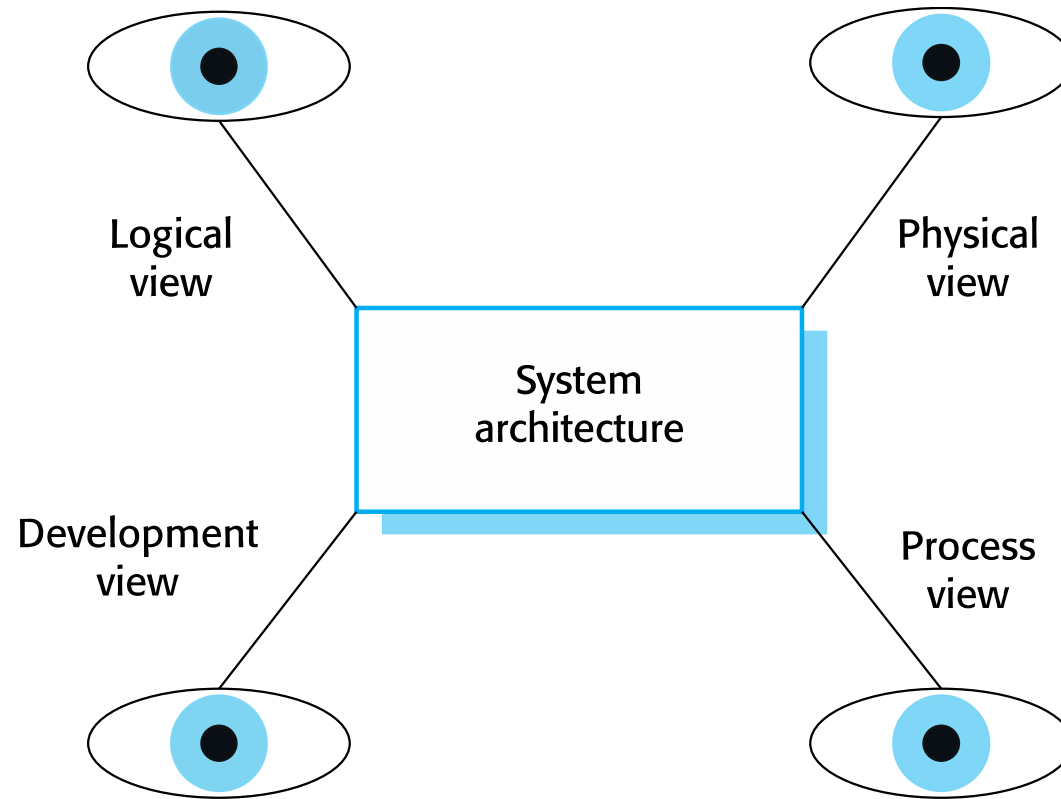
ARCHITECTURAL VIEWS



ARCHITECTURAL VIEWS

- ✓ What views or perspectives are useful when designing and documenting a system's architecture?
- ✓ What notations should be used for describing architectural models?
- ✓ Each architectural model only shows one view or perspective of the system.
 - how a system is decomposed into modules
 - how the run-time processes interact
 - system components are distributed across a network

ARCHITECTURAL VIEWS

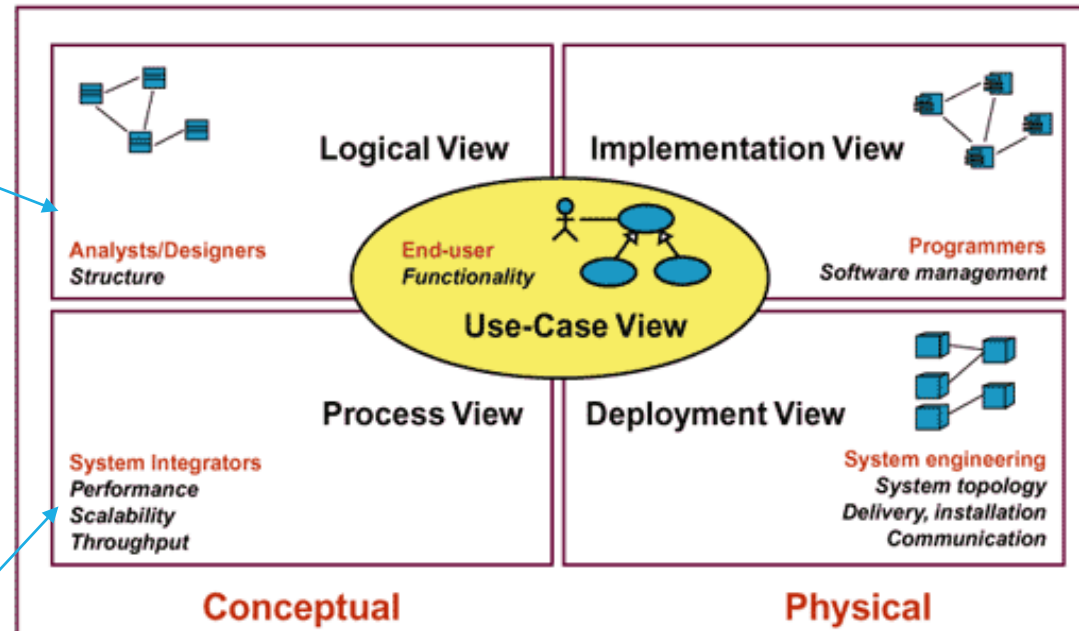


need to present multiple views of the software architecture.

4 + 1 VIEW MODEL OF SOFTWARE ARCHITECTURE

shows the key abstractions in the system as objects or classes.

shows how the software is decomposed for development.

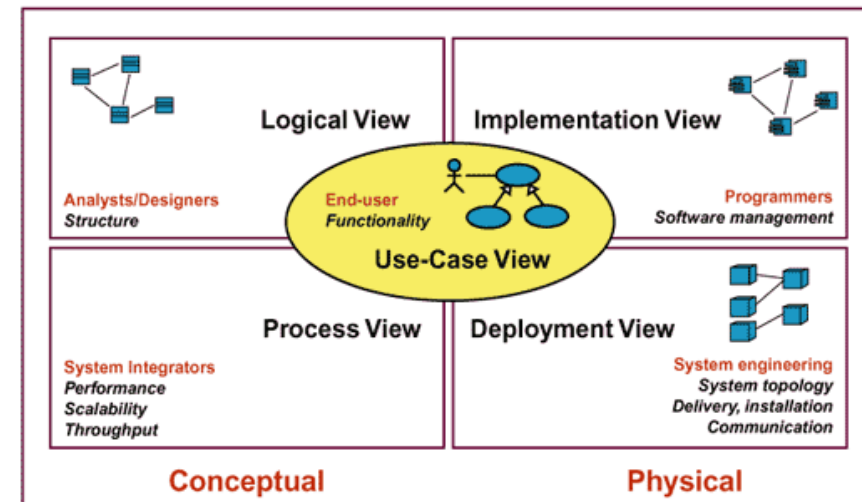


shows how, at run-time, the system is composed of interacting processes.

shows the system hardware and how software components are distributed across the processors in the system.

4 + 1 VIEW MODEL VS UML

- ✓ Logical view:
 - Class diagram, Communication diagram, Sequence diagram
- ✓ Process view:
 - Activity diagram
- ✓ Development view:
 - Component diagram, Package diagram.
- ✓ Physical view:
 - Deployment diagram
- ✓ Scenarios (+1):
 - Use-case





ARCHITECTURAL PATTERNS



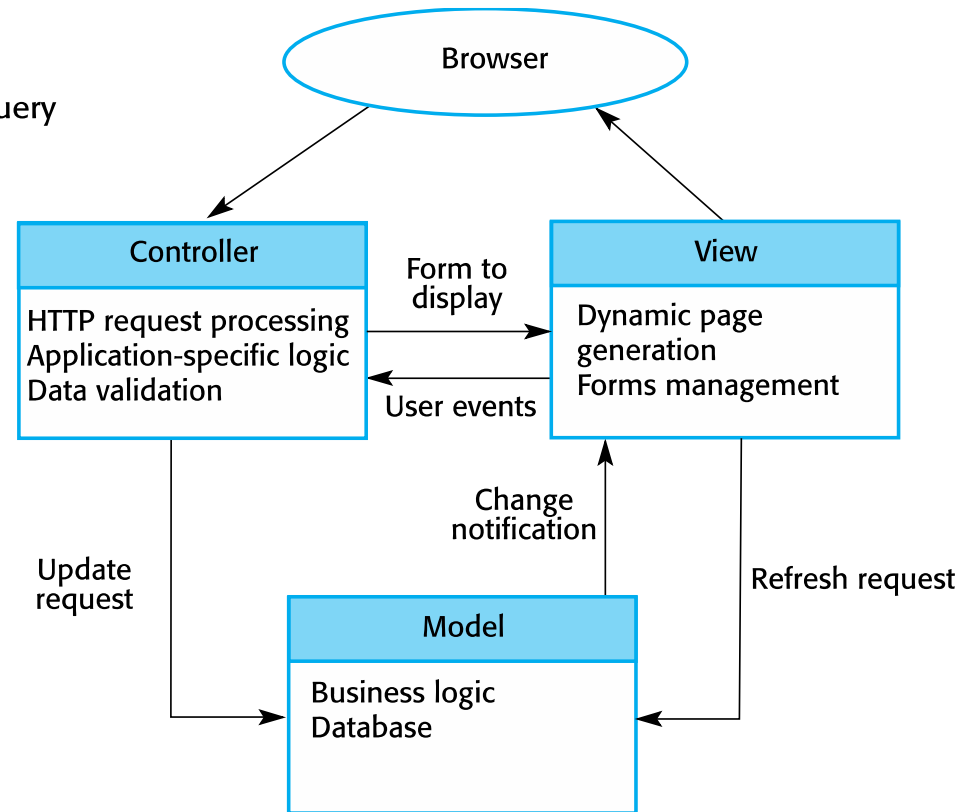
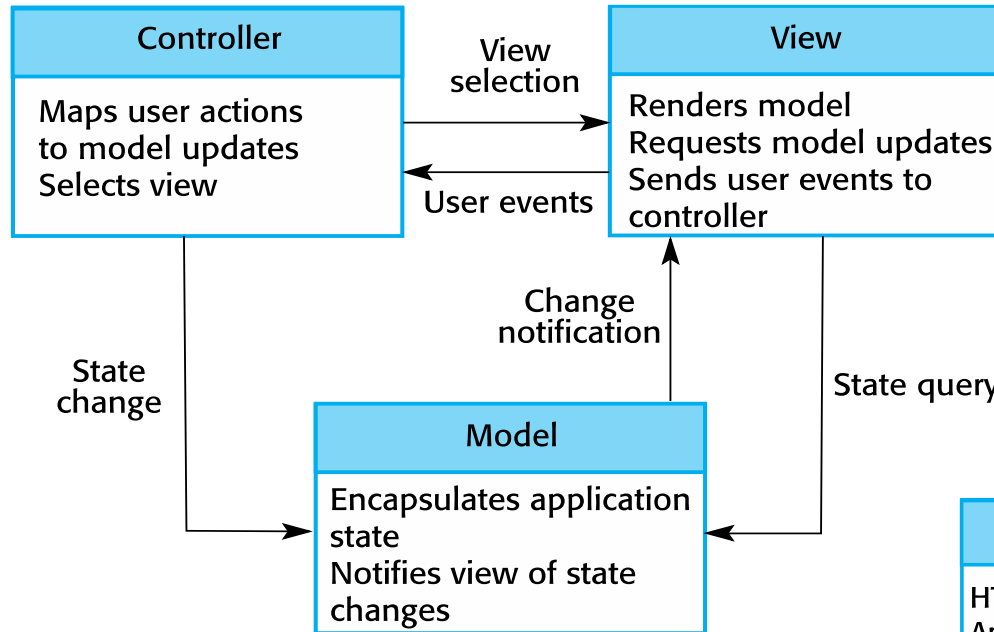
ARCHITECTURAL PATTERNS

- ✓ Patterns are a means of representing, sharing and reusing knowledge.
 - Patterns should include information about when they are and when they are not useful.
 - Patterns may be represented using tabular and graphical descriptions.
- ✓ An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.

THE MODEL-VIEW-CONTROLLER (MVC) PATTERN

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.
Example	The next slide shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

CONCEPTUAL VIEW OF THE MVC AND WEB-BASED MVC

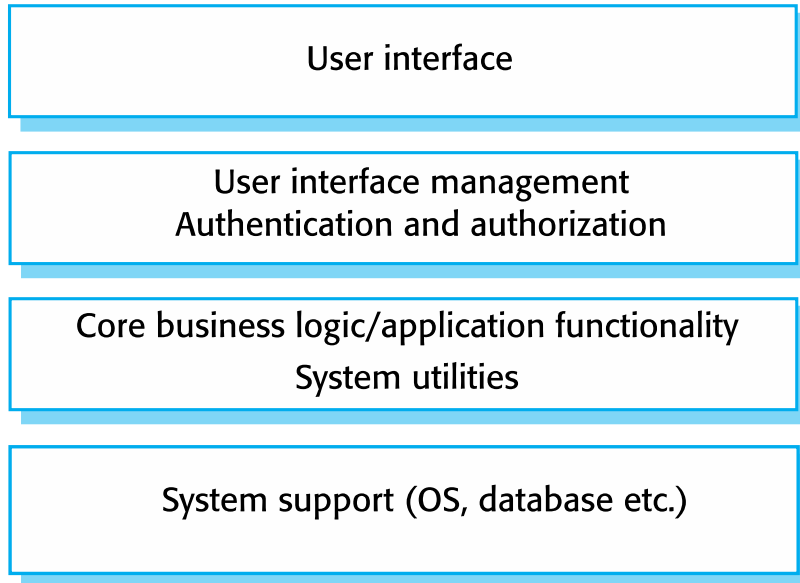


THE LAYERED ARCHITECTURE PATTERN

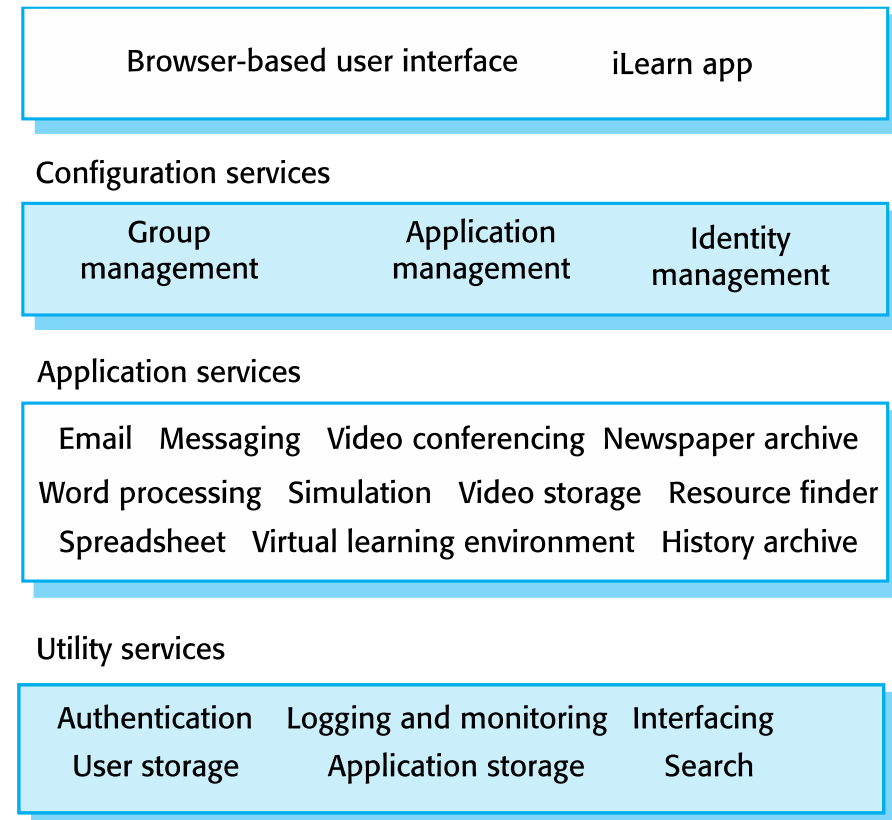
Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system.
Example	A layered model of a system for sharing copyright documents held in different libraries
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

A GENERIC LAYERED ARCHITECTURE

A generic layered architecture



The architecture of the iLearn system



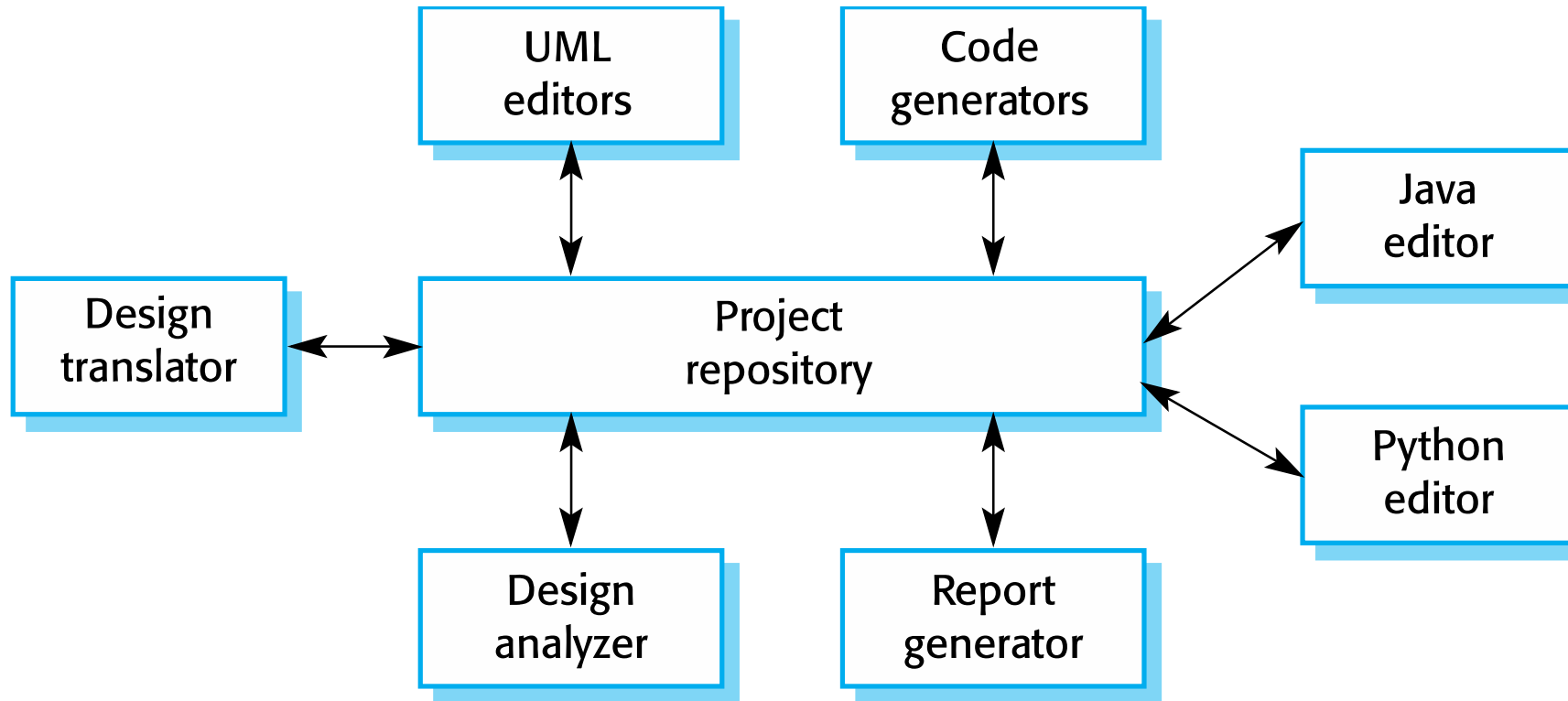
REPOSITORY ARCHITECTURE

- ✓ Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems;
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- ✓ When large amounts of data are to be shared, the repository model of sharing is most commonly used as this is an efficient data sharing mechanism.

THE REPOSITORY PATTERN

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	The next is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

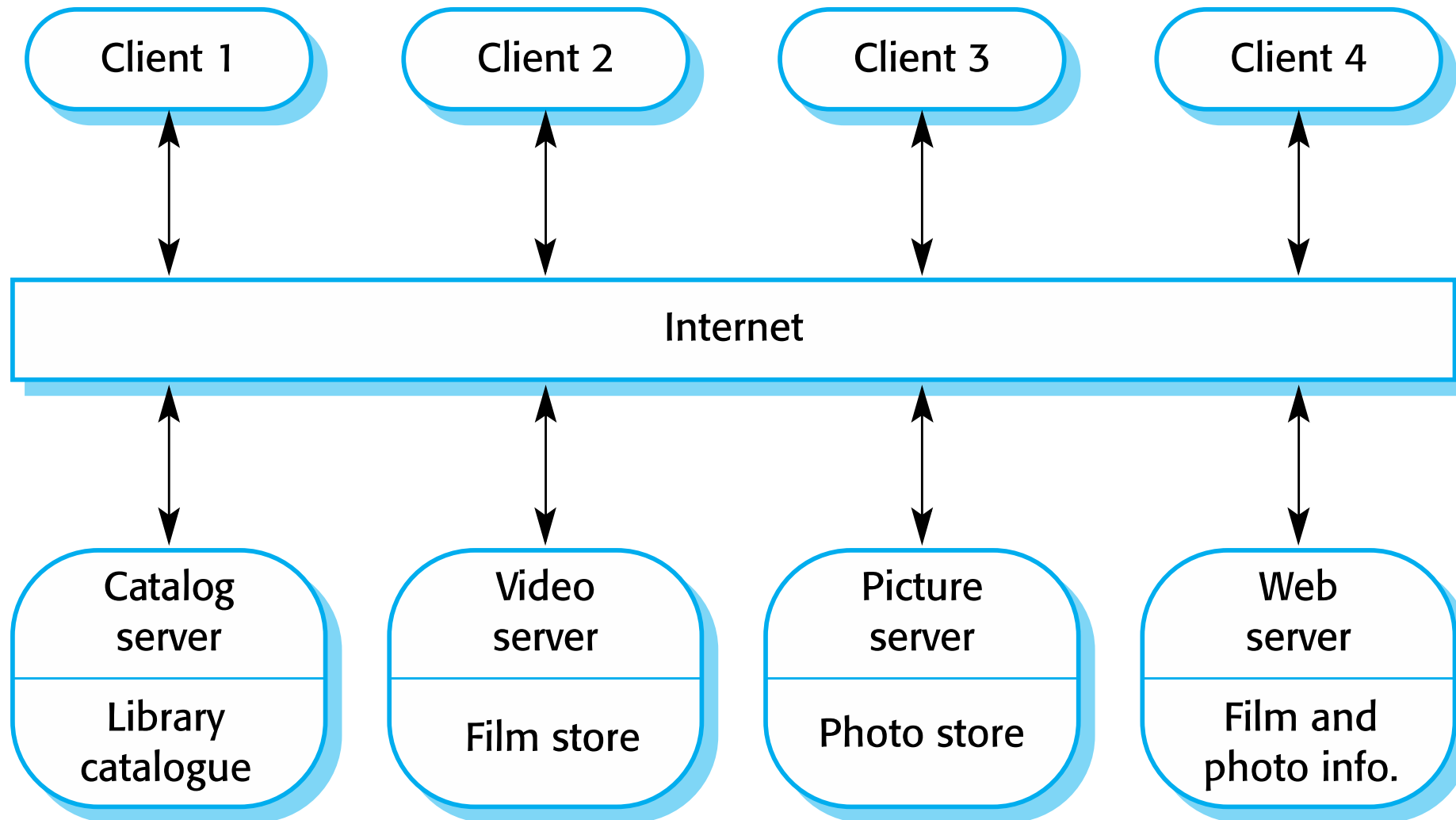
A REPOSITORY ARCHITECTURE FOR AN IDE



THE CLIENT—SERVER PATTERN

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure in the next slide is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

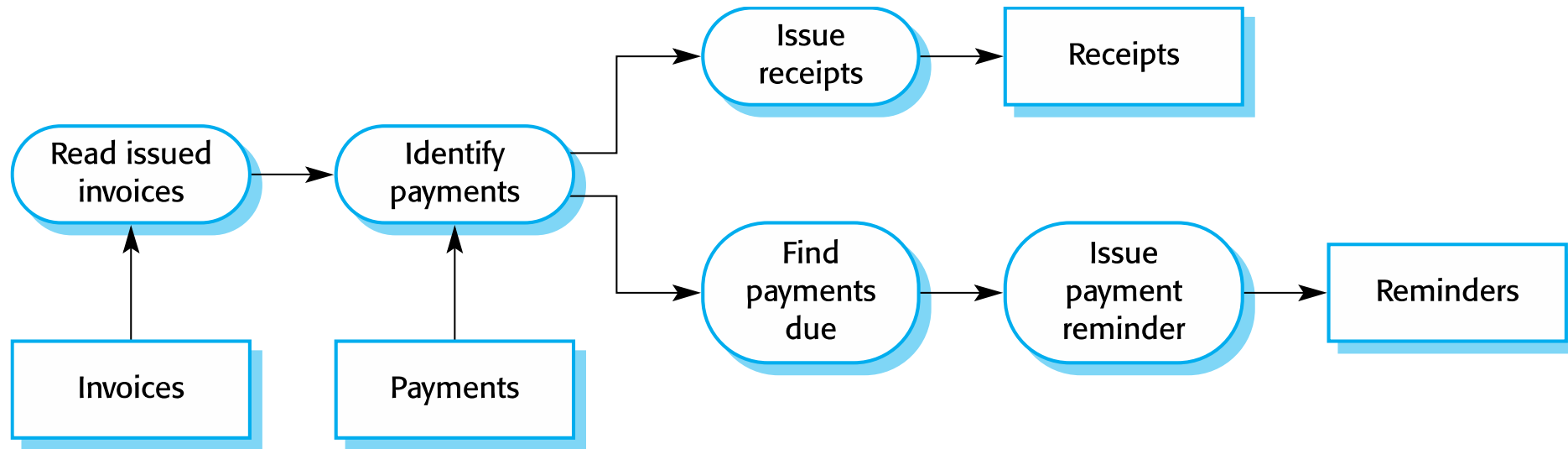
A CLIENT—SERVER ARCHITECTURE FOR A FILM LIBRARY



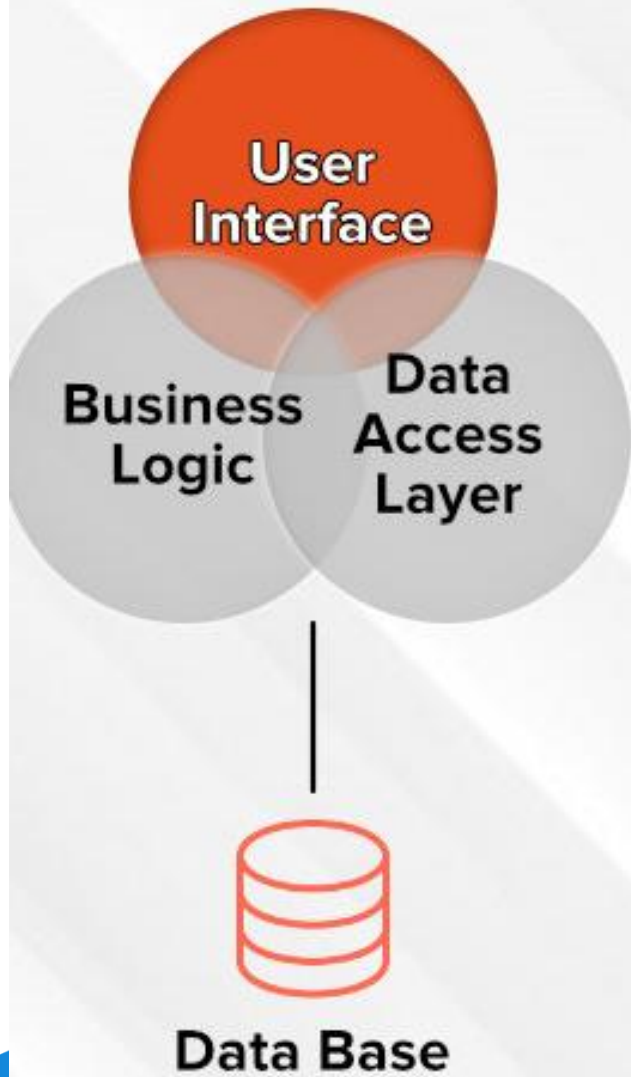
THE PIPE AND FILTER PATTERN

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure in the next slide is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

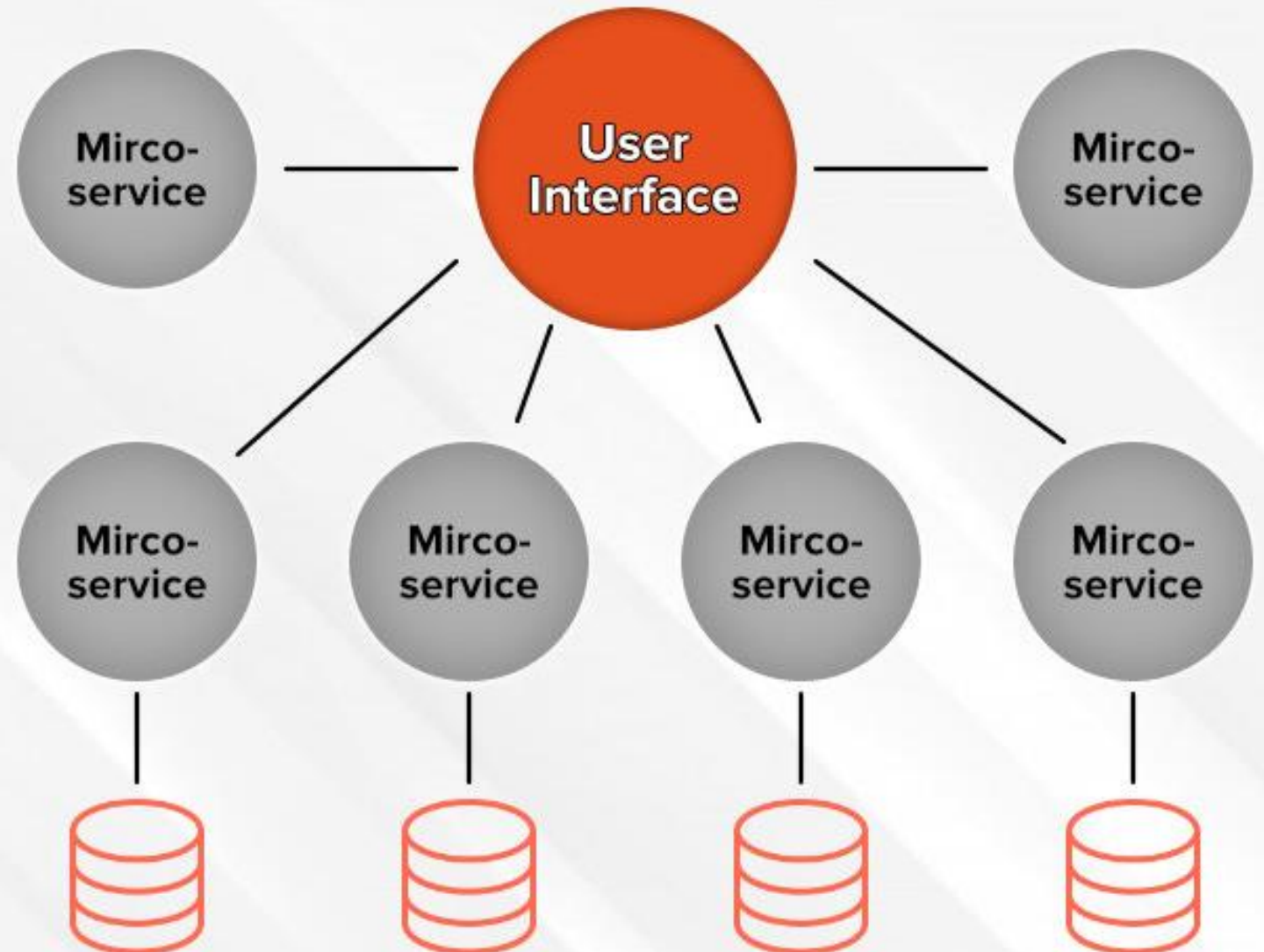
AN EXAMPLE OF THE PIPE AND FILTER ARCHITECTURE



MONOLITHIC ARCHITECTURE

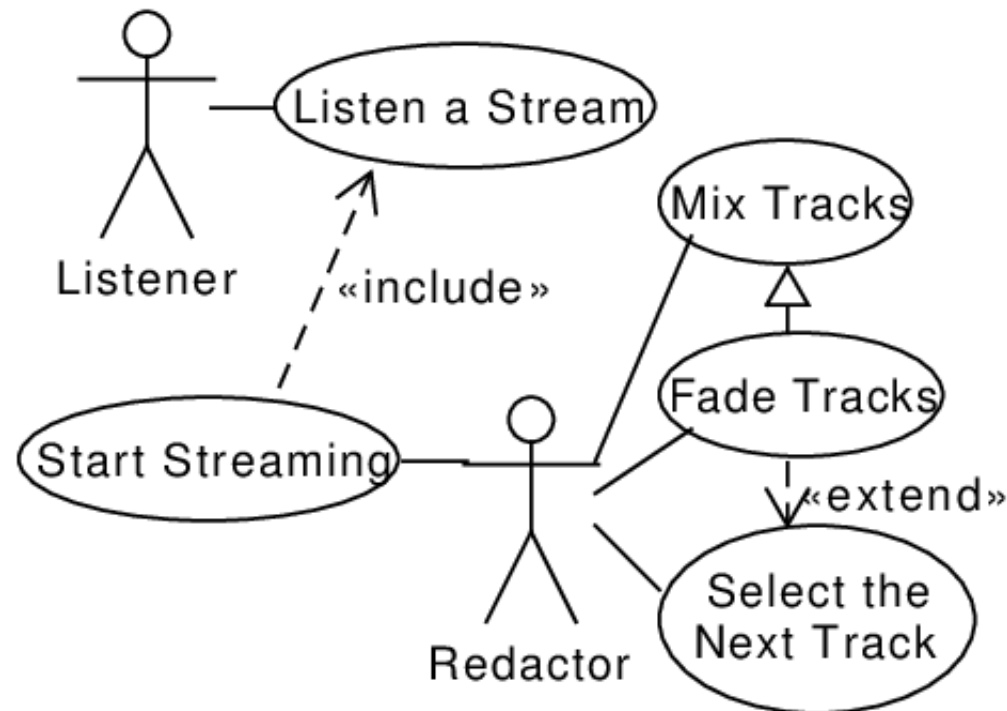


MICROSERVICE ARCHITECTURE



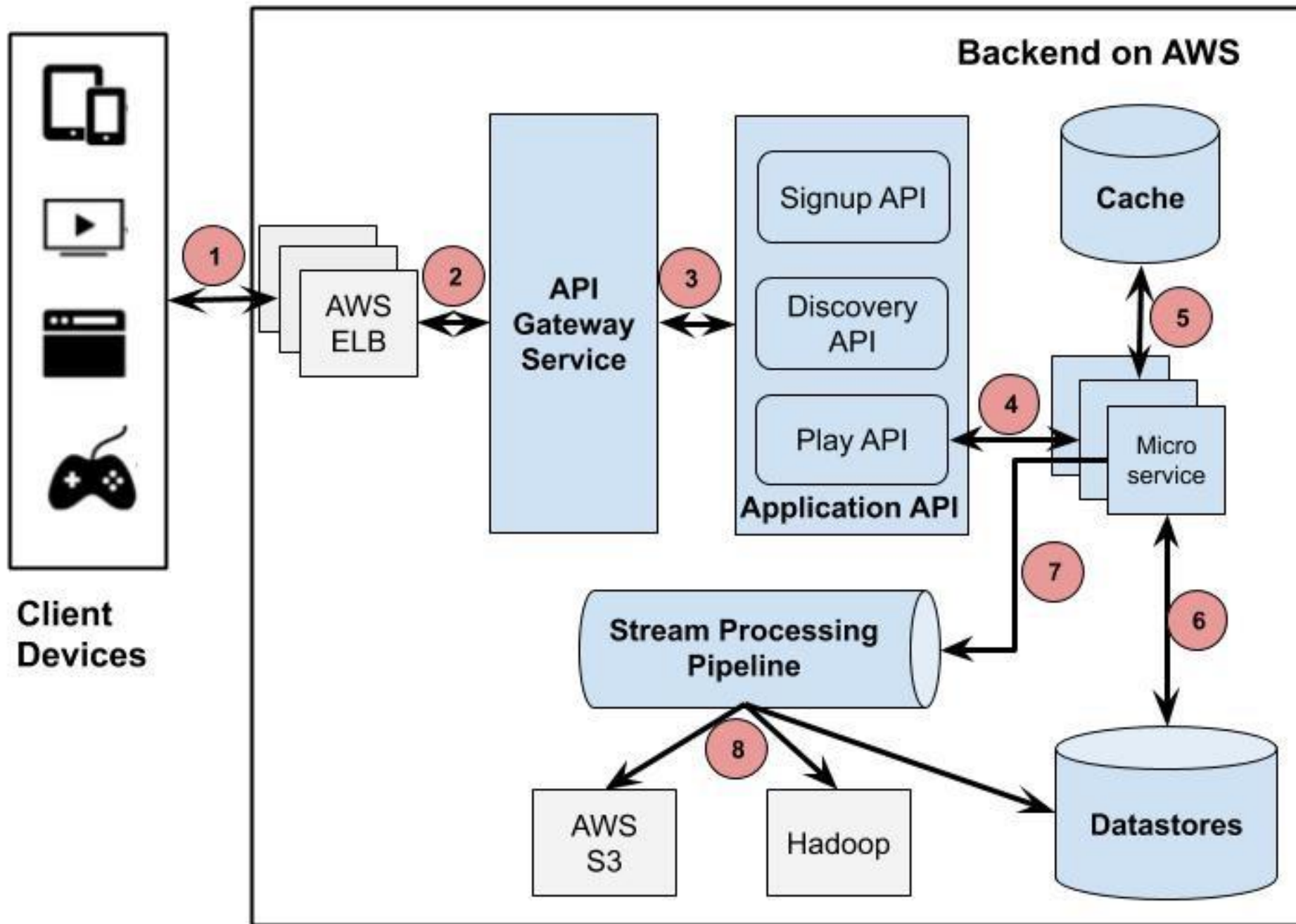
ARCHITECTURE FOR A STREAMING PLATFORM

- ✓ a streaming platform is an on-demand online entertainment source for TV shows, movies and other streaming media. For example, think of things like Hulu, Netflix, Amazon Prime Video, Vimeo, and Sundance Now.



READ ABOUT

- ✓ Client-server pattern
- ✓ Repository pattern
- ✓ Layered pattern
- ✓ MVC pattern
- ✓ Pipe and filter pattern
- ✓ Microservice vs. Monolithic architecture
 - Pros & cons



Monolithic architecture	Microservices architecture
Simple to develop and deploy	Complex and hard to develop and deploy
Interconnectivity	Autonomy
Code changes affect the entire system	Only the microservice that is changed would be affected
Inter-service communication	Using APIs for communication
Simple testing The entire system will be tested at once.	Testing is much more complex Done on a per unit or component level after which a system-wide test can then be done
One codebase and one shared database	A codebase and database for each microservice
Hard to scale or upgrade	Very scalable and upgrade
Less expensive and faster to develop	More expensive and takes more time to develop
The entire system can be affected by a single error or bug	The entire system is shielded from the error or bug on one sub service



APPLICATION ARCHITECTURES



APPLICATION ARCHITECTURES

- ✓ Application systems are designed to meet an organizational need.
 - As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements.
- ✓ A generic application architecture is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.

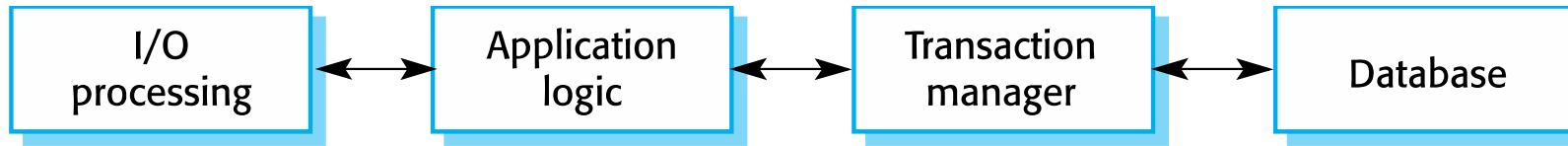
USE OF APPLICATION ARCHITECTURES

- ✓ As a starting point for architectural design.
- ✓ As a design checklist.
- ✓ As a way of organising the work of the development team.
- ✓ As a means of assessing components for reuse.
- ✓ As a vocabulary for talking about application types.

EXAMPLES OF APPLICATION TYPES

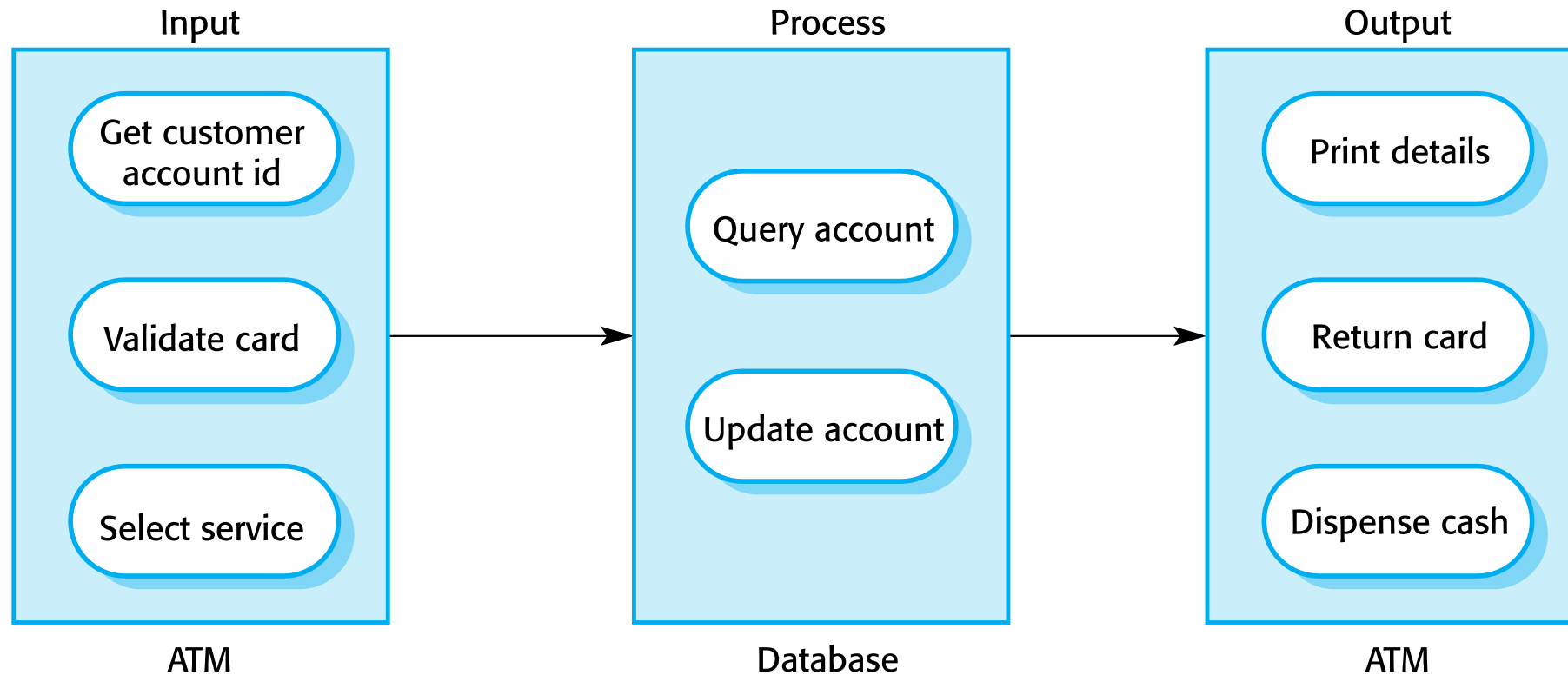
- ✓ Transaction processing applications
 - Database-centred applications that process user requests and update information in a system database.
- ✓ Event processing systems
 - Applications where system actions depend on interpreting events from the system's environment.
- ✓ Language processing systems
 - Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.

TRANSACTION PROCESSING SYSTEMS



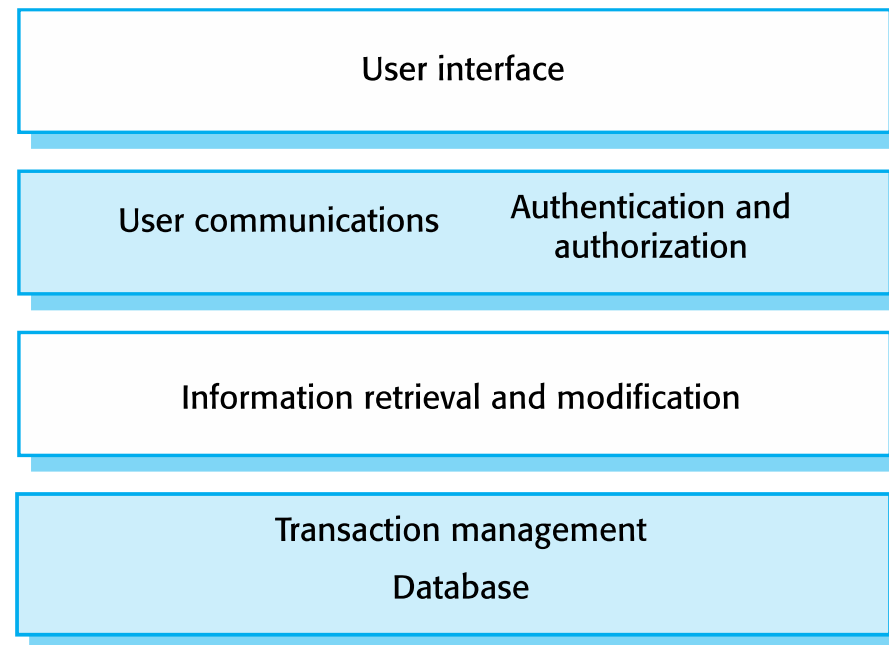
- ✓ Process user requests for information from a database or requests to update the database.
- ✓ From a user perspective a transaction is:
 - Any coherent sequence of operations that satisfies a goal;
 - For example - find the times of flights from London to Paris.
- ✓ Users make asynchronous requests for service which are then processed by a transaction manager.

THE SOFTWARE ARCHITECTURE OF AN ATM SYSTEM

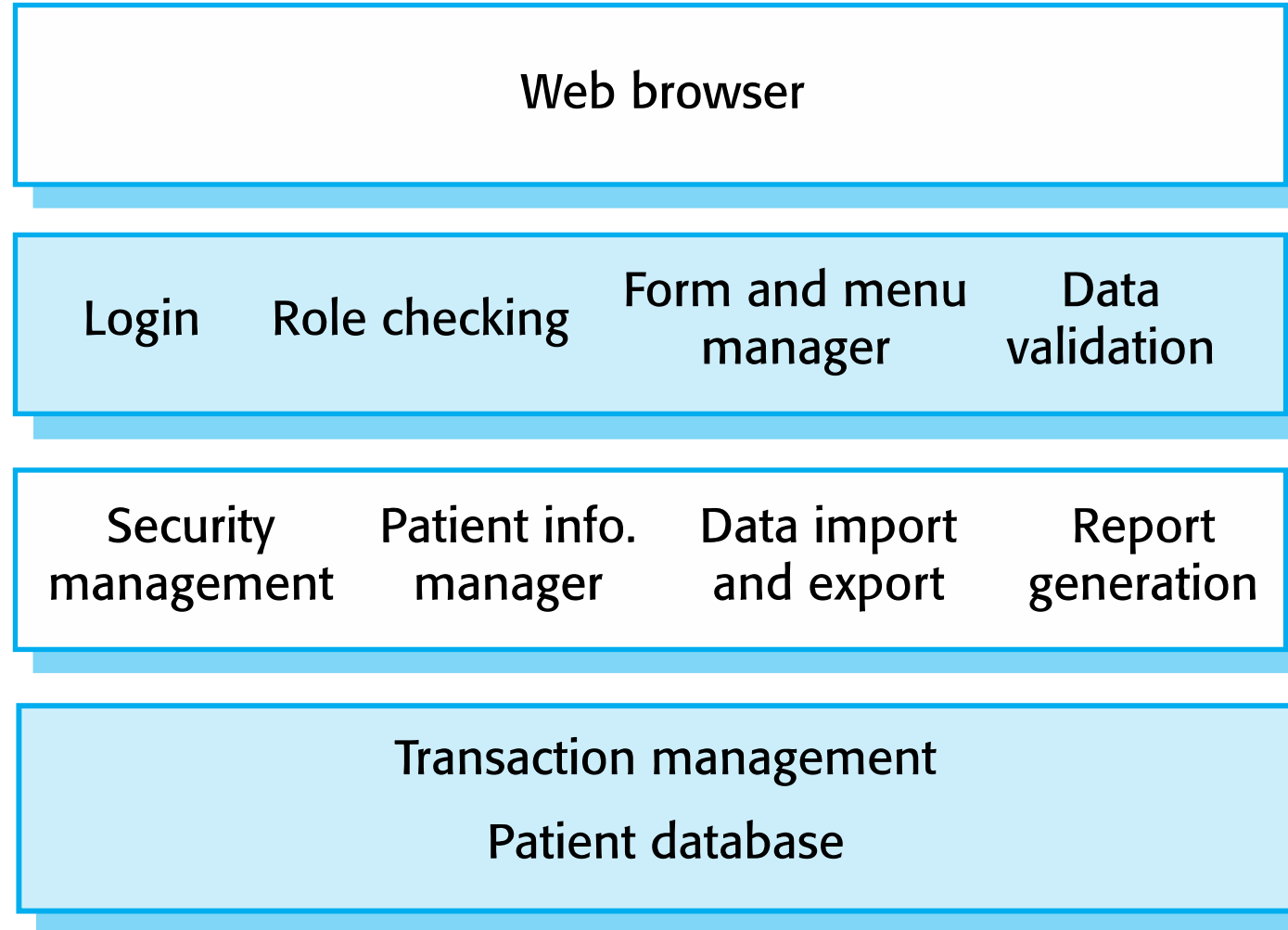


INFORMATION SYSTEMS ARCHITECTURE

- ✓ Information systems have a generic architecture that can be organized as a layered architecture.
- ✓ These are transaction-based systems as interaction with these systems generally involves database transactions.
- ✓ Layers include:
 - The user interface
 - User communications
 - Information retrieval
 - System database



THE ARCHITECTURE OF THE MENTCARE SYSTEM



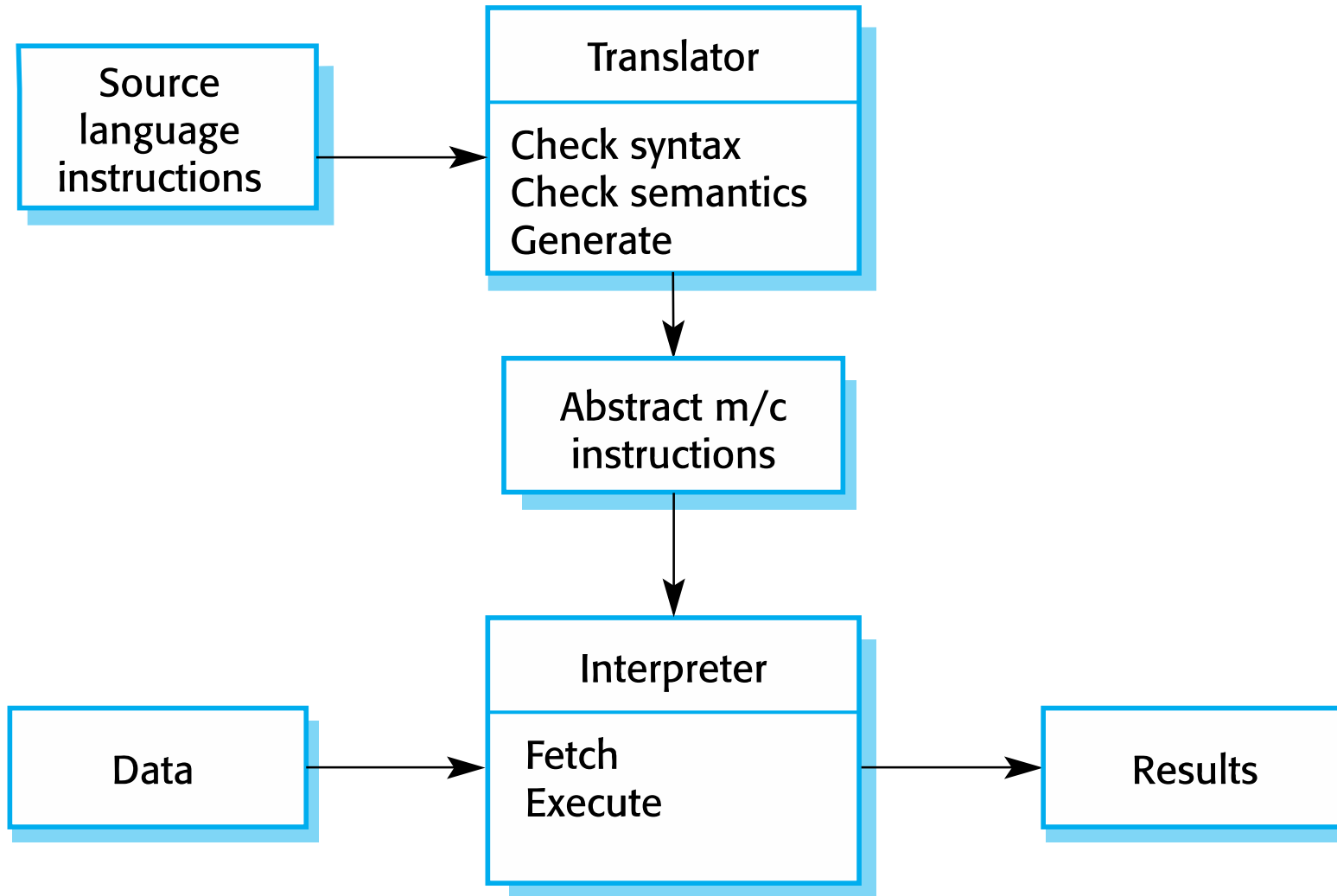
WEB-BASED INFORMATION SYSTEMS

- ✓ Information and resource management systems are now usually web-based systems where the user interfaces are implemented using a web browser.
- ✓ Often implemented as multi-tier client/server architectures.
 - The web server is responsible for all user communications, with the user interface implemented using a web browser;
 - The application server is responsible for implementing application-specific logic as well as information storage and retrieval requests;
 - The database server moves information to and from the database and handles transaction management.

LANGUAGE PROCESSING SYSTEMS

- ✓ Accept a natural or artificial language as input and generate some other representation of that language.
- ✓ May include an interpreter to act on the instructions in the language that is being processed.
- ✓ Used in situations where the easiest way to solve a problem is to describe an algorithm or describe the system data
 - Meta-case tools process tool descriptions, method rules, etc and generate tools.

THE ARCHITECTURE OF A LANGUAGE PROCESSING SYSTEM

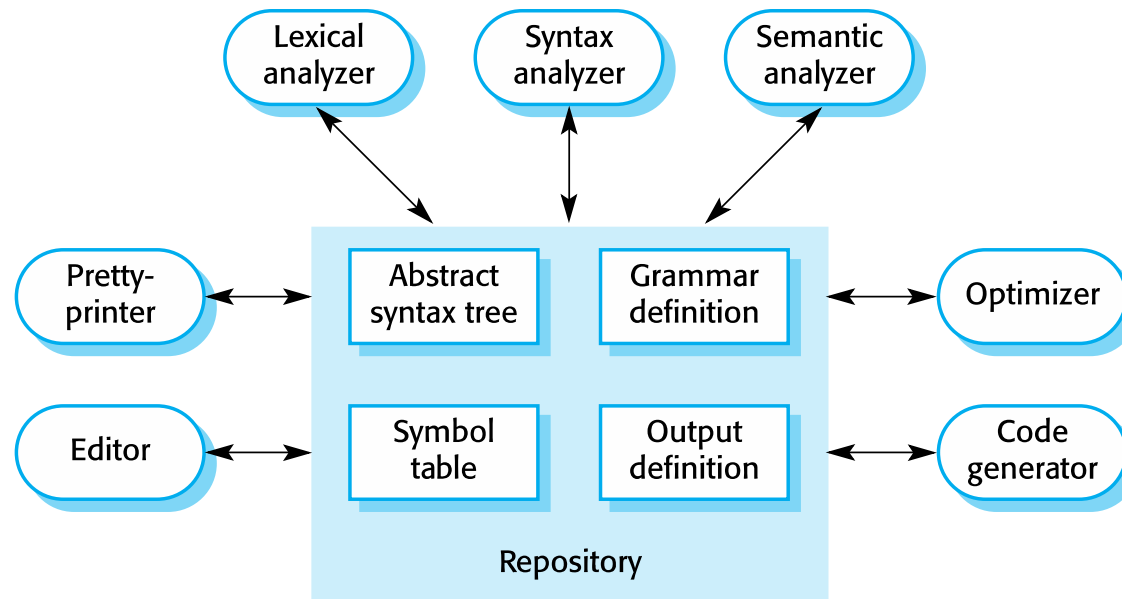
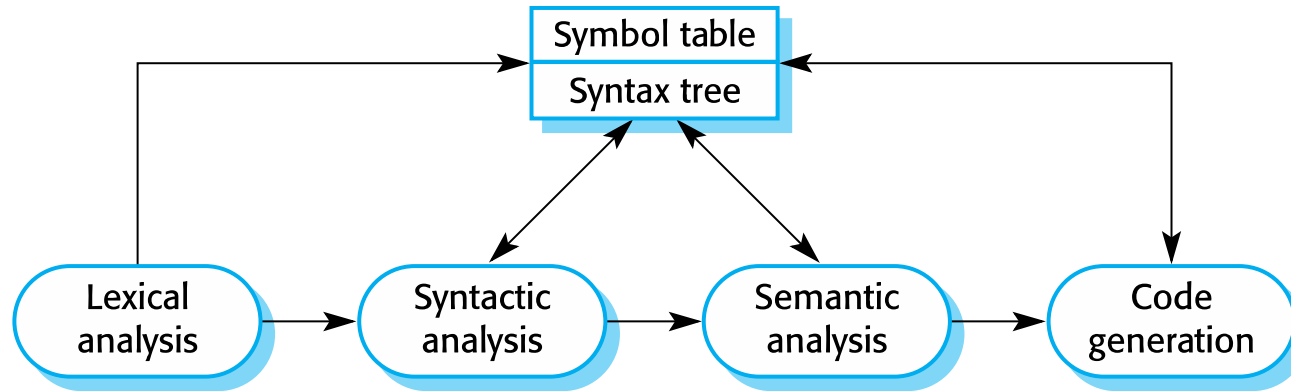


COMPILER ARCHITECTURE

✓ Components

- A lexical analyzer: takes input language tokens and converts them to an internal form.
- A symbol table: holds information about the names of entities (variables, class names, object names, etc.) used in the text that is being translated.
- A syntax analyzer: checks the syntax of the language being translated.
- A syntax tree: an internal structure representing the program being compiled.
- A semantic analyzer: uses information from the syntax tree and the symbol table to check the semantic correctness of the input language text.
- A code generator: 'walks' the syntax tree and generates abstract machine code.

A PIPE AND FILTER VS REPOSITORY ARCHITECTURE FOR COMPILERS



SUMMARY

- ✓ A software architecture is a description of how a software system is organized.
- ✓ Architectural design decisions include decisions on the type of application, the distribution of the system, the architectural styles to be used.
- ✓ Architectures may be documented from several different perspectives or views such as a conceptual view, a logical view, a process view, and a development view.
- ✓ Architectural patterns are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used and describe its advantages and disadvantages.

SUMMARY (CONT.)

- ✓ Models of application systems architectures help us understand and compare applications, validate application system designs and assess large-scale components for reuse.
- ✓ Transaction processing systems are interactive systems that allow information in a database to be remotely accessed and modified by a number of users.
- ✓ Language processing systems are used to translate texts from one language into another and to carry out the instructions specified in the input language. They include a translator and an abstract machine that executes the generated language.